

analise_treinamento_parafusos_yolo11

June 3, 2026

1 Analise completa - Parafusos com YOLO11

Notebook para verificar o dataset, visualizar anotacoes, treinar/validar YOLO11 e analisar resultados de contagem.

Por padrao ele usa dataset_detect, que e a versao convertida para bounding boxes.

```
[1]: from pathlib import Path
import sys, random, csv
from datetime import datetime

PROJECT_DIR = Path.cwd()
if PROJECT_DIR.name == "notebooks":
    PROJECT_DIR = PROJECT_DIR.parent

DATASET_DIR = PROJECT_DIR / "dataset_detect"
DATA_YAML = DATASET_DIR / "data.yaml"
MODEL_NAME = "yolo11_screws_hard_100_gpu" # modelo bom: 100 epocas. Evite
↳ yolo11_screws, que treinou so 1 epoca.
MODEL_PATH = PROJECT_DIR / "runs_screws" / MODEL_NAME / "weights" / "best.pt"
OUTPUT_DIR = PROJECT_DIR / "outputs" / "notebook_analysis"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

print("Projeto:", PROJECT_DIR)
print("Dataset:", DATASET_DIR, DATASET_DIR.exists())
print("data.yaml:", DATA_YAML, DATA_YAML.exists())
print("Modelo ativo:", MODEL_NAME)
print("Modelo:", MODEL_PATH, MODEL_PATH.exists())
if MODEL_PATH.exists():
    print("Tamanho do modelo MB:", round(MODEL_PATH.stat().st_size / 1024 /
↳ 1024, 2))
assert MODEL_NAME == "yolo11_screws_hard_100_gpu", "Voce esta usando o modelo
↳ antigo. Rode esta celula de configuracao novamente."
```

Projeto: c:\Users\Carla Batista\Desktop\desafio-1

Dataset: c:\Users\Carla Batista\Desktop\desafio-1\dataset_detect True

data.yaml: c:\Users\Carla Batista\Desktop\desafio-1\dataset_detect\data.yaml

True

Modelo ativo: yolo11_screws_hard_100_gpu

Modelo: c:\Users\Carla
Batista\Desktop\desafio-1\runs_screws\yolo11_screws_hard_100_gpu\weights\best.pt
True
Tamanho do modelo MB: 18.44

1.1 1. Ambiente

```
[2]: print("Python:", sys.version)
try:
    import ultralytics
    print("ultralytics:", ultralytics.__version__)
except Exception as exc:
    print("ultralytics nao instalado:", exc)

try:
    import cv2
    print("OpenCV:", cv2.__version__)
except Exception as exc:
    print("OpenCV nao instalado:", exc)

try:
    import torch
    print("PyTorch:", torch.__version__)
    print("CUDA:", torch.cuda.is_available())
    if torch.cuda.is_available():
        print("GPU:", torch.cuda.get_device_name(0))
except Exception as exc:
    print("PyTorch nao disponivel:", exc)
```

Python: 3.12.13 | packaged by Anaconda, Inc. | (main, Mar 19 2026, 20:16:45)
[MSC v.1942 64 bit (AMD64)]
ultralytics: 8.4.56
OpenCV: 4.11.0
PyTorch: 2.11.0+cu128
CUDA: True
GPU: NVIDIA GeForce RTX 4050 Laptop GPU

Se faltar dependencia, rode no terminal: `pip install -r requirements.txt`.

1.2 2. Dataset e estatísticas

```
[3]: IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".webp"}

def split_dirs(split):
    return DATASET_DIR / "images" / split, DATASET_DIR / "labels" / split

def list_images(split):
    image_dir, _ = split_dirs(split)
```

```

        return sorted([p for p in image_dir.iterdir() if p.suffix.lower() in
↪ IMAGE_EXTENSIONS]) if image_dir.exists() else []

def label_path_for(image_path, split):
    _, label_dir = split_dirs(split)
    return label_dir / f"{image_path.stem}.txt"

def read_yolo_boxes(label_path):
    boxes = []
    if not label_path.exists():
        return boxes
    for line_no, raw in enumerate(label_path.read_text(encoding="utf-8").
↪ splitlines(), start=1):
        parts = raw.strip().split()
        if not parts:
            continue
        if len(parts) != 5:
            raise ValueError(f"Linha mal formatada em {label_path}:{line_no}:
↪ {raw}")
        cls, xc, yc, w, h = parts
        boxes.append((int(float(cls)), float(xc), float(yc), float(w),
↪ float(h)))
    return boxes

rows, bad_files, classes = [], [], set()
for split in ["train", "val", "test"]:
    images = list_images(split)
    image_dir, label_dir = split_dirs(split)
    labels = sorted(label_dir.glob("*.txt")) if label_dir.exists() else []
    print(f"{split:5s} | imagens={len(images):5d} | labels={len(labels):5d}")
    for image in images:
        try:
            boxes = read_yolo_boxes(label_path_for(image, split))
            rows.append({"split": split, "imagem": image.name, "qtd_boxes":
↪ len(boxes)})
            classes.update([b[0] for b in boxes])
        except Exception as exc:
            bad_files.append((str(image), str(exc)))

print("Classes encontradas:", sorted(classes))
print("Arquivos com problema:", len(bad_files))

```

```

train | imagens= 2904 | labels= 2904
val   | imagens=  308 | labels=  308
test  | imagens=  119 | labels=  119
Classes encontradas: [0]
Arquivos com problema: 0

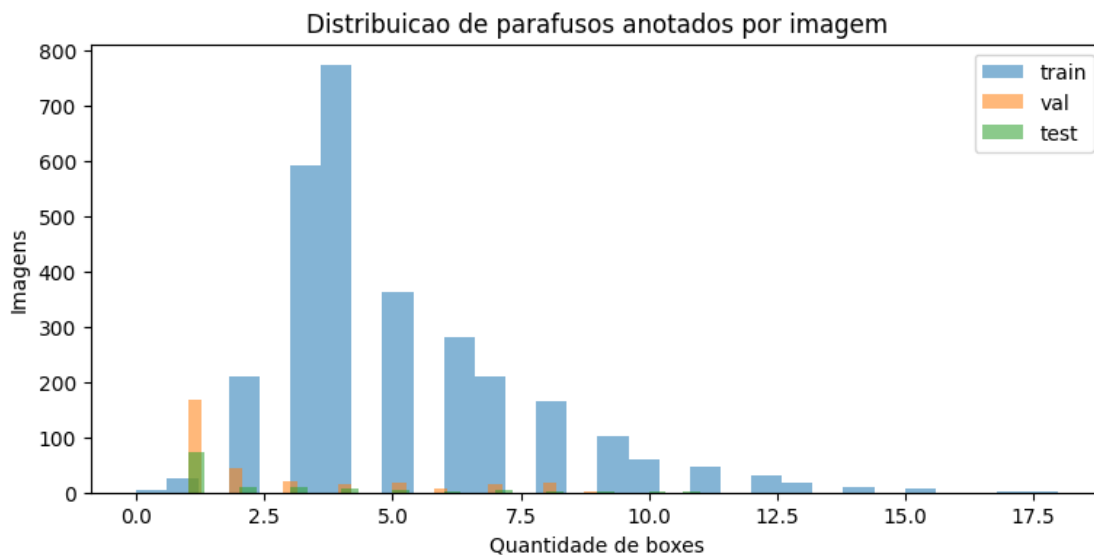
```

```
[5]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

df_counts = pd.DataFrame(rows)
display(df_counts.groupby("split")["qtd_boxes"].describe())

plt.figure(figsize=(9, 4))
for split in ["train", "val", "test"]:
    plt.hist(df_counts[df_counts["split"] == split]["qtd_boxes"], bins=30,
            alpha=0.55, label=split)
plt.title("Distribuicao de parafusos anotados por imagem")
plt.xlabel("Quantidade de boxes")
plt.ylabel("Imagens")
plt.legend()
plt.show()
```

	count	mean	std	min	25%	50%	75%	max
split								
test	119.0	2.386555	2.307355	1.0	1.0	1.0	3.0	11.0
train	2904.0	5.020317	2.474381	0.0	3.0	4.0	6.0	18.0
val	308.0	2.555195	2.288665	1.0	1.0	1.0	3.0	9.0



1.3 3. Visualizar anotacoes reais

```
[6]: import cv2

def draw_yolo_boxes(image_path, boxes, color=(20, 130, 255)):
```

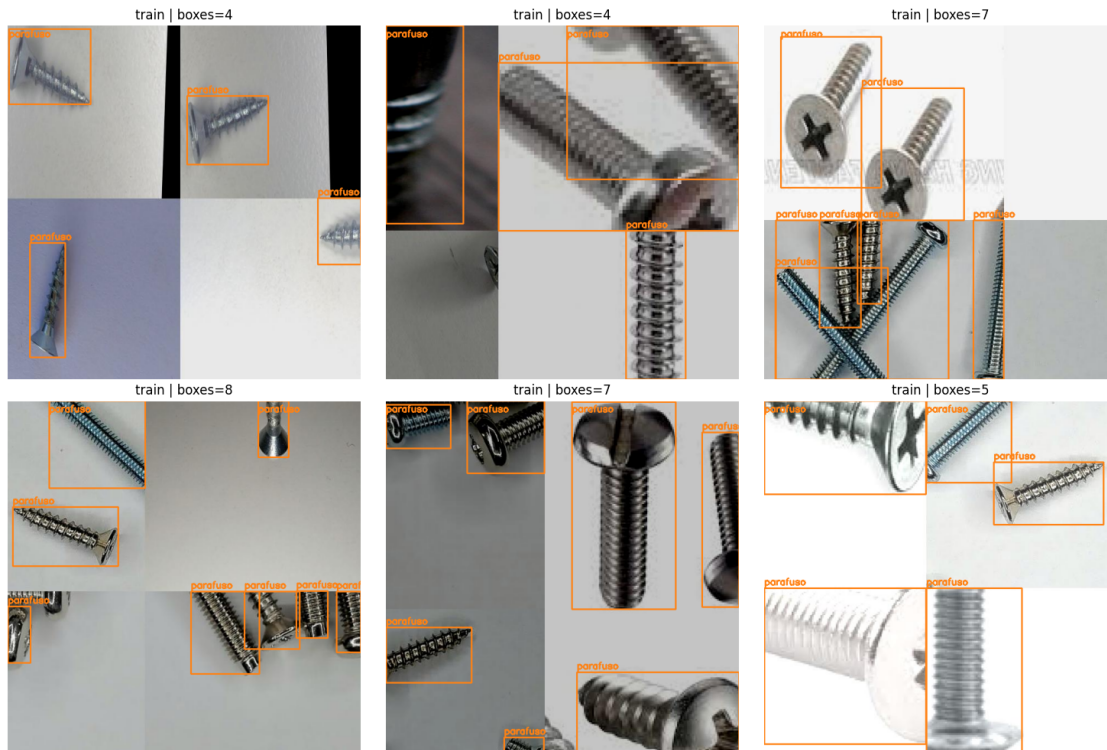
```

image = cv2.imread(str(image_path))
if image is None:
    raise ValueError(f"Nao abriu {image_path}")
h, w = image.shape[:2]
for cls, xc, yc, bw, bh in boxes:
    x1 = int((xc - bw / 2) * w)
    y1 = int((yc - bh / 2) * h)
    x2 = int((xc + bw / 2) * w)
    y2 = int((yc + bh / 2) * h)
    cv2.rectangle(image, (x1, y1), (x2, y2), color, 2)
    cv2.putText(image, "parafuso", (x1, max(20, y1 - 6)), cv2.
↳FONT_HERSHEY_SIMPLEX, 0.55, color, 2)
    return cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

def show_annotated_samples(split="train", n=6, seed=7):
    images = list_images(split)
    random.Random(seed).shuffle(images)
    samples = images[:n]
    cols = 3
    rows_plot = int(np.ceil(n / cols))
    plt.figure(figsize=(15, 5 * rows_plot))
    for i, image_path in enumerate(samples, start=1):
        boxes = read_yolo_boxes(label_path_for(image_path, split))
        plt.subplot(rows_plot, cols, i)
        plt.imshow(draw_yolo_boxes(image_path, boxes))
        plt.title(f"{split} | boxes={len(boxes)}")
        plt.axis("off")
    plt.tight_layout()
    plt.show()

show_annotated_samples("train", 6)

```



1.4 4. Treinar YOLO11

```
[7]: RUN_TRAINING = False
BASE_MODEL = "yolo11s.pt" # yolo11n.pt e mais rapido; yolo11s.pt tende a ser
    ↪ mais preciso
EPOCHS = 100
IMGSZ = 960 # 640 e mais rapido; 960/1280 ajuda em objetos pequenos
BATCH = 8

if RUN_TRAINING:
    from ultralytics import YOLO
    import torch
    device = 0 if torch.cuda.is_available() else "cpu"
    model = YOLO(BASE_MODEL)
    model.train(
        data=str(DATA_YAML), epochs=EPOCHS, imgsz=IMGSZ, batch=BATCH,
        patience=30, workers=4, device=device,
        project=str(PROJECT_DIR / "runs_screws"), name="yolo11_screws",
        ↪ close_mosaic=10,
    )
else:
    print("Treinamento desativado. Altere RUN_TRAINING para True para treinar.")
```

Treinamento desativado. Altere RUN_TRAINING para True para treinar.

1.5 5. Graficos do treinamento

```
[8]: RUN_DIR = PROJECT_DIR / "runs_screws" / "yolo11_screws"
results_csv = RUN_DIR / "results.csv"
results_png = RUN_DIR / "results.png"

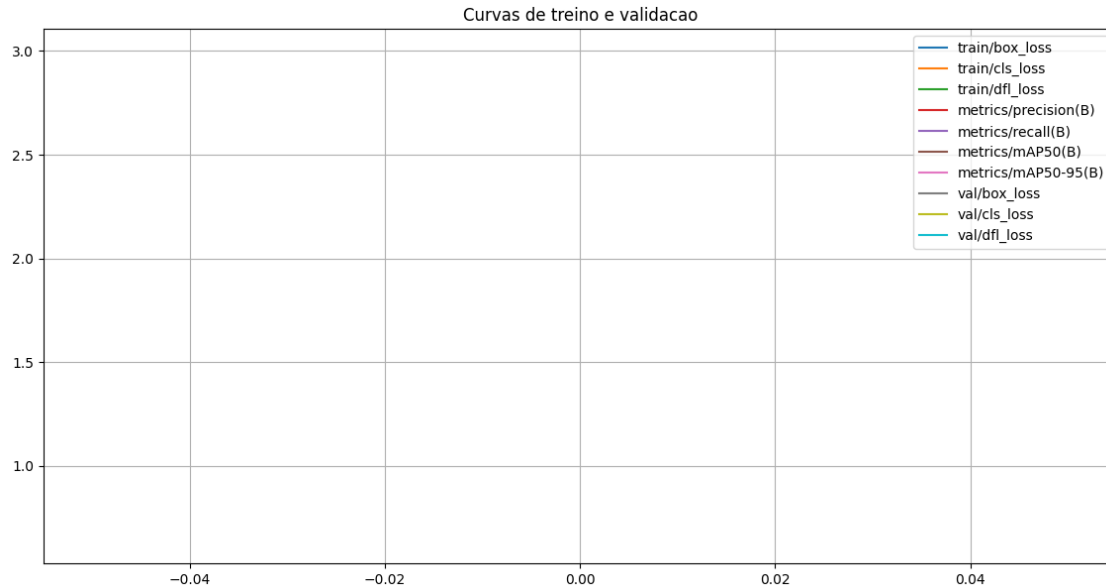
if results_csv.exists():
    train_df = pd.read_csv(results_csv)
    display(train_df.tail())
    cols = [c for c in train_df.columns if "metrics" in c or "loss" in c]
    train_df[cols].plot(figsize=(14, 7), grid=True)
    plt.title("Curvas de treino e validacao")
    plt.show()
elif results_png.exists():
    img = cv2.cvtColor(cv2.imread(str(results_png)), cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(14, 8))
    plt.imshow(img)
    plt.axis("off")
    plt.show()
else:
    print("Ainda nao ha resultados de treino para exibir.")
```

epoch	time	train/box_loss	train/cls_loss	train/df1_loss	\
0	1	3349.92	0.87846	1.24353	1.17219

	metrics/precision(B)	metrics/recall(B)	metrics/mAP50(B)	\
0	0.82937	0.7751	0.87356	

	metrics/mAP50-95(B)	val/box_loss	val/cls_loss	val/df1_loss	lr/pg0	\
0	0.64686	1.24015	2.98885	1.84925	0.000665	

	lr/pg1	lr/pg2
0	0.000665	0.000665



1.6 6. Validacao do modelo

```
[9]: if MODEL_PATH.exists():
    from ultralytics import YOLO
    model = YOLO(str(MODEL_PATH))
    metrics = model.val(data=str(DATA_YAML), conf=0.25)
    print(f"precision: {float(metrics.box.mp):.4f}")
    print(f"recall: {float(metrics.box.mr):.4f}")
    print(f"mAP50: {float(metrics.box.map50):.4f}")
    print(f"mAP50-95: {float(metrics.box.map):.4f}")
else:
    print("Modelo ainda nao encontrado. Treine primeiro ou ajuste MODEL_PATH.")
```

Ultralytics 8.4.56 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 4050 Laptop GPU, 6140MiB)

YOLO11s summary (fused): 101 layers, 9,413,187 parameters, 0 gradients, 21.3 GFLOPs

val: Fast image access (ping: 0.10.0 ms, read: 84.237.6 MB/s, size: 27.7 KB)

val: Scanning C:\Users\Carla

Batista\Desktop\desafio-1\dataset_detect\labels\val.cache... 308 images, 0 backgrounds, 0 corrupt: 100% 308/308 0.0s

	Class	Images	Instances	Box(P	R	mAP50
mAP50-95): 100%		20/20	1.9s/it	37.1s	1.5ss	
	all	308	787	0.866	0.872	0.931

0.759

Speed: 22.2ms preprocess, 90.1ms inference, 0.0ms loss, 1.7ms postprocess per image

Results saved to C:\Users\Carla

Batista\Desktop\desafio-1\notebooks\runs\detect\val-6

precision: 0.8662

recall: 0.8721

mAP50: 0.9309

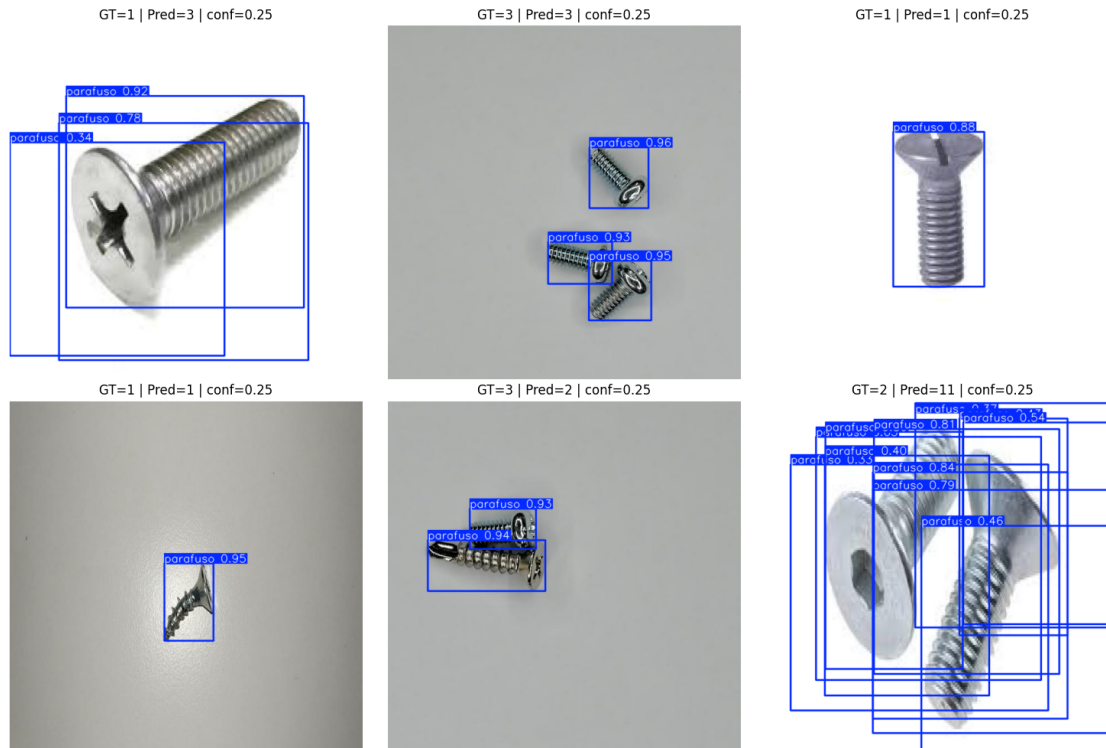
mAP50-95: 0.7591

Precision alta indica menos falsos parafusos. Recall alto indica menos parafusos perdidos. Para contagem, os dois precisam ficar equilibrados.

1.7 7. Inferencia visual em imagens de teste

```
[11]: def run_predictions(split="test", n=6, conf=0.25, seed=11):
    if not MODEL_PATH.exists():
        print("Modelo nao encontrado:", MODEL_PATH)
        return
    from ultralytics import YOLO
    model = YOLO(str(MODEL_PATH))
    images = list_images(split)
    random.Random(seed).shuffle(images)
    samples = images[:n]
    cols = 3
    rows_plot = int(np.ceil(n / cols))
    plt.figure(figsize=(15, 5 * rows_plot))
    for i, image_path in enumerate(samples, start=1):
        result = model.predict(str(image_path), conf=conf, verbose=False)[0]
        plotted = cv2.cvtColor(result.plot(), cv2.COLOR_BGR2RGB)
        pred_count = 0 if result.bboxes is None else len(result.bboxes)
        true_count = len(read_yolo_boxes(label_path_for(image_path, split)))
        plt.subplot(rows_plot, cols, i)
        plt.imshow(plotted)
        plt.title(f"GT={true_count} | Pred={pred_count} | conf={conf}")
        plt.axis("off")
    plt.tight_layout()
    plt.show()

run_predictions("test", n=6, conf=0.25)
```



1.8 8. Varredura de confiança

```
[12]: def confidence_sweep(split="test", max_images=80, conf_values=(0.15, 0.20, 0.
↪25, 0.30, 0.40)):
    if not MODEL_PATH.exists():
        print("Modelo nao encontrado:", MODEL_PATH)
        return pd.DataFrame()
    from ultralytics import YOLO
    model = YOLO(str(MODEL_PATH))
    images = list_images(split)[:max_images]
    out = []
    for conf in conf_values:
        errors = []
        for image_path in images:
            gt = len(read_yolo_boxes(label_path_for(image_path, split)))
            result = model.predict(str(image_path), conf=conf, verbose=False)[0]
            pred = 0 if result.bboxes is None else len(result.bboxes)
            errors.append(pred - gt)
        out.append({
            "conf": conf,
            "imagens": len(images),
            "erro_abs_medio": float(np.mean(np.abs(errors))) if errors else 0,
            "erro_medio_assinado": float(np.mean(errors)) if errors else 0,
```

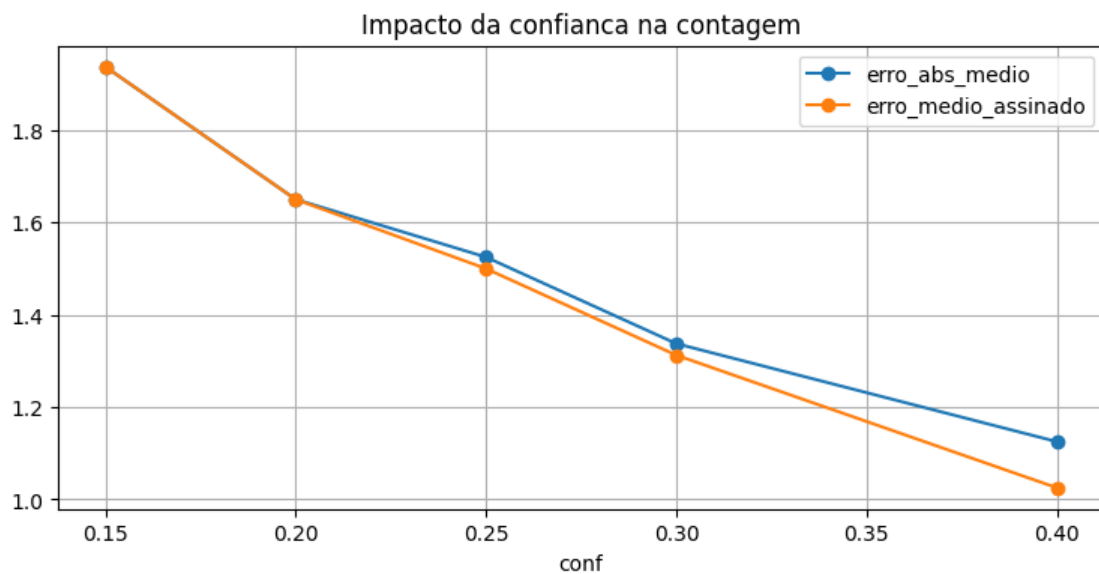
```

        "erro_abs_total": int(np.sum(np.abs(errors))) if errors else 0,
    })
    return pd.DataFrame(out)

sweep_df = confidence_sweep()
display(sweep_df)
if not sweep_df.empty:
    sweep_df.plot(x="conf", y=["erro_abs_medio", "erro_medioassinado"],
        marker="o", grid=True, figsize=(9, 4))
    plt.title("Impacto da confianca na contagem")
    plt.show()

```

	conf	imagens	erro_abs_medio	erro_medioassinado	erro_abs_total
0	0.15	80	1.9375	1.9375	155
1	0.20	80	1.6500	1.6500	132
2	0.25	80	1.5250	1.5000	122
3	0.30	80	1.3375	1.3125	107
4	0.40	80	1.1250	1.0250	90



Se o erro medio assinado for negativo, o modelo esta perdendo parafusos: reduza a confianca. Se for positivo, ele tende a contar falsos: aumente a confianca.

1.9 9. CSV de contagens em lote

```

[13]: def batch_count_to_csv(split="test", conf=0.25, max_images=100):
    if not MODEL_PATH.exists():
        print("Modelo nao encontrado:", MODEL_PATH)
        return None

```

```

from ultralytics import YOLO
model = YOLO(str(MODEL_PATH))
images = list_images(split)[:max_images]
csv_path = OUTPUT_DIR / f"contagens_{split}_conf_{conf}.csv"
with csv_path.open("w", newline="", encoding="utf-8") as file:
    writer = csv.DictWriter(file, fieldnames=["image", "gt", "pred",
↪"erro", "conf", "data_hora"])
    writer.writeheader()
    for image_path in images:
        gt = len(read_yolo_boxes(label_path_for(image_path, split)))
        result = model.predict(str(image_path), conf=conf, verbose=False)[0]
        pred = 0 if result.bboxes is None else len(result.bboxes)
        writer.writerow({"image": image_path.name, "gt": gt, "pred": pred,
↪"erro": pred - gt, "conf": conf, "data_hora": datetime.now().
↪isoformat(timespec="seconds")})
    return csv_path

csv_path = batch_count_to_csv()
print("CSV gerado:", csv_path)

```

CSV gerado: c:\Users\Carla

Batista\Desktop\desafio-1\outputs\notebook_analysis\contagens_test_conf_0.25.csv

1.10 10. Teste da funcao modular usada pelo backend

```

[ ]: if str(PROJECT_DIR) not in sys.path:
    sys.path.insert(0, str(PROJECT_DIR))

from screw_counter import contar_parafusos

if MODEL_PATH.exists():
    sample = list_images("test")[0]
    result = contar_parafusos(sample, MODEL_PATH, OUTPUT_DIR, conf=0.25,
↪funcionario_id="notebook", funcionario_nome="Teste Notebook",
↪csv_path=OUTPUT_DIR / "contagens_funcao_modular.csv")
    display(result)
    img = cv2.cvtColor(cv2.imread(result["imagem_resultado"]), cv2.
↪COLOR_BGR2RGB)
    plt.figure(figsize=(10, 8))
    plt.imshow(img)
    plt.axis("off")
    plt.show()
else:
    print("Modelo nao encontrado. Esta celula fica disponivel apos o
↪treinamento.")

```

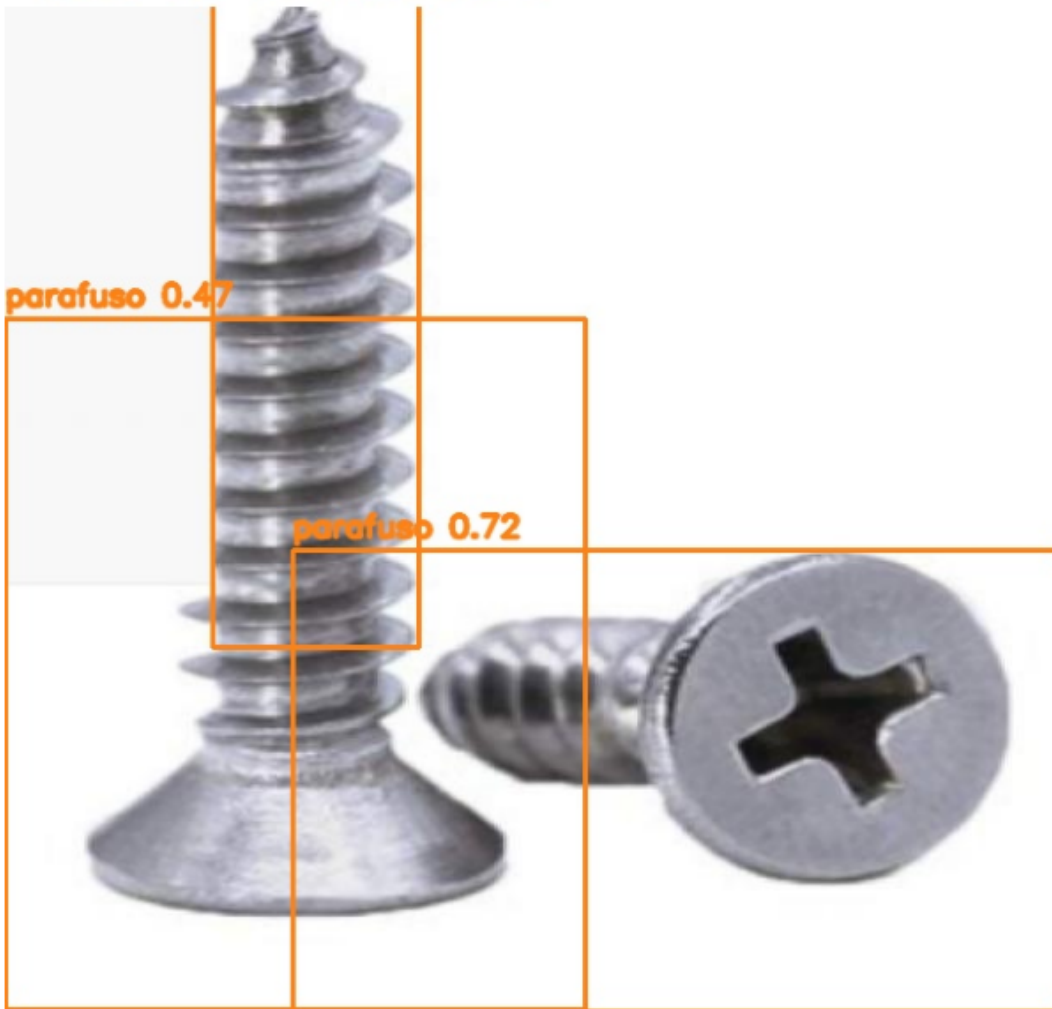
```

{'funcionario_id': 'notebook',
 'funcionario_nome': 'Teste Notebook',

```

```
'imagem': 'c:\\Users\\Carla_\\Batista\\Desktop\\desafio-1\\dataset_detect\\images\\test\\-2022-11-26-233904_png.rf.5a6dfb10717953e92a860abfa0ebac63.jpg',  
'total_parafusos': 3,  
'confianca_media': 0.4926,  
'imagem_resultado': 'c:\\Users\\Carla_\\Batista\\Desktop\\desafio-1\\outputs\\notebook_analysis\\-2022-11-26-233904_png.rf.5a6dfb10717953e92a860abfa0ebac63_20260529_085139_090630.jpg',  
'data_hora': '2026-05-29T08:51:39'}
```

Total de parafusos: 3



The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the `↩failure`.

Click

View Jupyter

1.11 Checklist final

- Verifique visualmente falsos positivos e falsos negativos.
- Teste fotos reais do smartphone.
- Ajuste `CONF_THRESHOLD` no backend conforme a varredura.
- Para objetos pequenos, teste `imgsz=960` ou `1280`.
- Retreine com exemplos reais ruins: reflexos, baixa luz, fundo variado e sobreposicao.